JAVA COURSE

{ ereditarietà }



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.1

INDICE

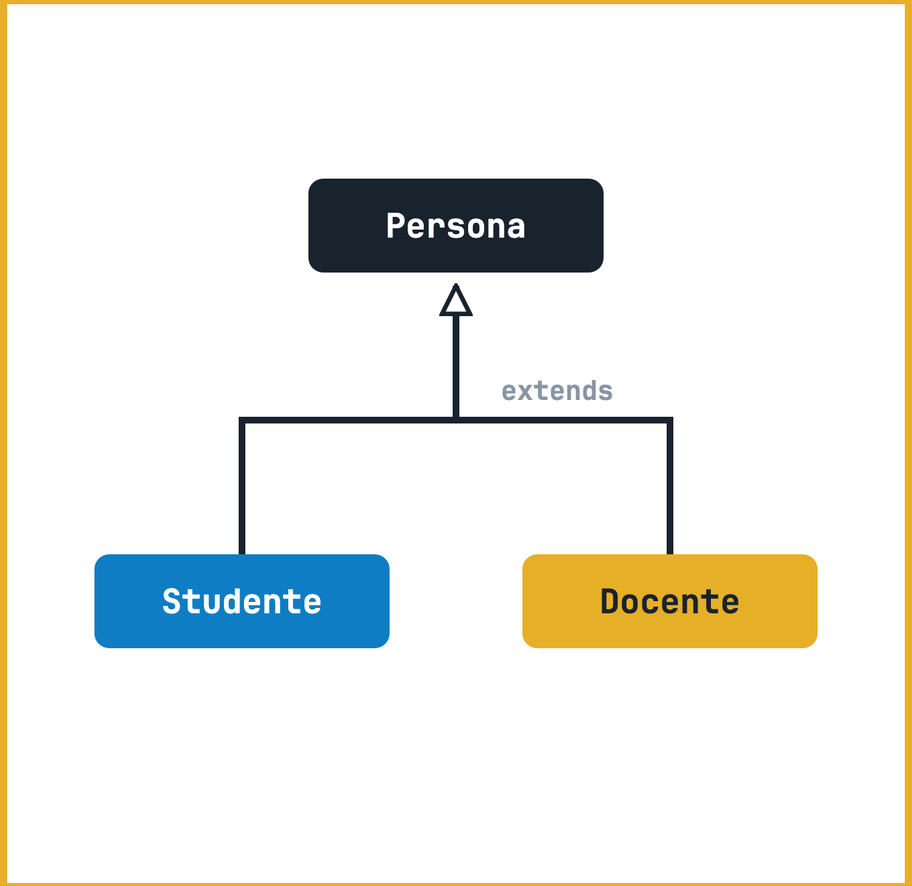
01 Cos'è l'ereditarietà

02 Costruttori e super

03 Override e Object

04 Ereditarietà in pratica

01
COS'È L'EREDITARIETÀ



IL PROBLEMA: CODICE DUPLICATO

Due classi che condividono gli stessi campi e metodi: nome, email, getter, toString

- `nome` e `email` *sono identici in entrambe le classi*
- *Se cambia la logica comune, bisogna modificare entrambe*
- *Il codice duplicato è fonte di bug e incoerenze*

```
class Studente {
    private String nome;
    private String email;
    private String matricola;
    // getter, toString ...
}

class Docente {
    private String nome;
    private String email;
    private String materia;
    // getter, toString ...
}
```

LA SOLUZIONE: EREDITARIETÀ

Si estrae il codice comune in una **classe base** e le classi specifiche la **estendono**

- `Studente` e `Docente` *hanno* `nome` e `email`*, scritti solo in* `Persona`
- *Ogni sottoclasse aggiunge solo i propri campi specifici*
- *Il codice comune è scritto una sola volta*

```
class Persona {
    private String nome;
    private String email;
    // getter, toString
}

class Studente extends Persona {
    private String matricola;
}

class Docente extends Persona {
    private String materia;
}
```

LA PAROLA CHIAVE EXTENDS

Per dichiarare che una classe eredita da un'altra si usa `extends`

- *La sottoclasse eredita i membri* `public` *e* `protected`
- *I campi* `private` *non si ereditano: esistono nell'oggetto, ma la sottoclasse non li vede*
- *Ai campi* `private` *si arriva con i metodi ereditati, come i getter*
- *I costruttori* ***non*** *vengono ereditati*

```
class Studente extends Persona {
    // Studente riceve i membri non private di Persona
    // e aggiunge i propri membri
}
```

TERMINOLOGIA

Tre coppie di termini equivalenti per descrivere la relazione

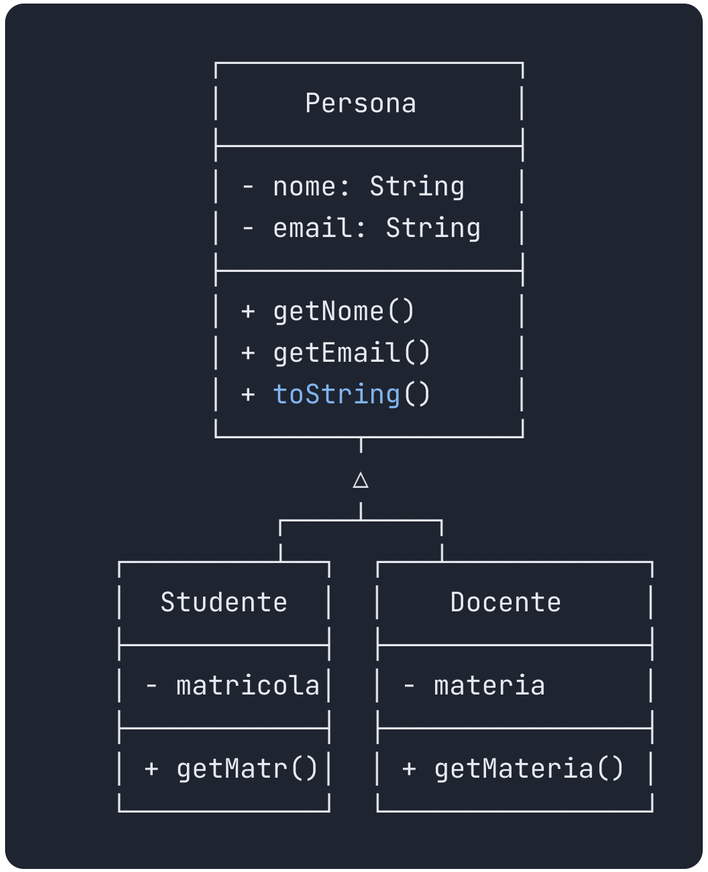
- *In Java si dice più spesso **superclasse** e **sottoclasse***
- *La sottoclasse **è un tipo più specifico** della superclasse*
- *La superclasse **è un tipo più generale***

Classe che dà	Classe che riceve
Superclasse	Sottoclasse
Classe padre	Classe figlia
Classe base	Classe derivata

DIAGRAMMA UML

Una freccia a **triangolo vuoto** va dalla sottoclasse alla superclasse

- *La freccia punta **verso la superclasse***
- *I campi* `-` *sono private, i metodi* `+` *sono public*



COSA SI EREDITA

Un riepilogo di ciò che la sottoclasse riceve dalla superclasse

Membro	Ereditato	Accessibile
Campi `public`	Sì	Sì
Campi `protected`	Sì	Sì
Campi `private`	No	No (esistono nell'oggetto, si usa il getter)
Metodi `public`	Sì	Sì
Metodi `protected`	Sì	Sì
Metodi `private`	No	No
Costruttori	No	No (si invocano con super)

SINGLE INHERITANCE

In Java una classe può estendere **una sola** classe

- *Java non supporta l'ereditarietà multipla di classi*
- *In Python, invece, una classe può estendere più classi*
- *Java offre le **interfacce** come alternativa (argomento futuro)*
- *Questa scelta evita ambiguità (il "problema del diamante")*

```
// corretto: una sola superclasse
class Studente extends Persona { }

// ERRORE: ereditarietà multipla non consentita
class StudLav extends Studente, Lavoratore { }
```

OGNI CLASSE ESTENDE OBJECT

Se una classe non usa `extends`, Java la fa estendere automaticamente da `Object`

- `Object` *è la radice di* ***tutte*** *le gerarchie in Java*
- *Ogni oggetto ha i metodi di Object:* `toString()`, `equals()`, *ecc.*
- *Per questo* `System.out.println(obj)` *funziona sempre*

```
// queste due dichiarazioni sono equivalenti
class Persona { }
class Persona extends Object { }
```

QUIZ

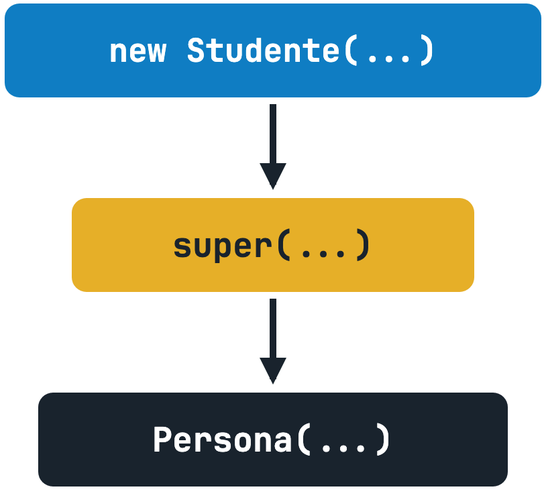
Quali affermazioni sono vere?

- *A) Una sottoclasse eredita i costruttori della superclasse*
- *B) Una classe Java può estendere al massimo una classe*
- *C) I campi* `private` *della superclasse non esistono negli oggetti della sottoclasse*
- *D) Se non scrivo* `extends`*, la mia classe estende* `Object`
- *E) La freccia UML dell'ereditarietà va dalla superclasse alla sottoclasse*

RISPOSTE

- *A) **Falso** — i costruttori non vengono ereditati; si invocano con `super()`*
- *B) **Vero** — Java supporta solo ereditarietà singola*
- *C) Falso — il campo esiste nell'oggetto, ma non è ereditato e la sottoclasse non può accedervi*
- *D) **Vero** — `Object` è la superclasse implicita*
- *E) **Falso** — la freccia va dalla sottoclasse alla superclasse*

02
COSTRUTTORI E SUPER



prima il genitore, poi il figlio

IL COSTRUTTORE DELLA SOTTOCLASSE

La sottoclasse **non eredita** i costruttori: deve definirne uno proprio e invocare quello della superclasse

- *La sottoclasse deve inizializzare i campi ereditati*
- *Ma quei campi sono* `private`*: non può accedervi direttamente*
- *La soluzione:* ***delegare al costruttore della superclasse***

```
class Persona {
    private String nome;
    private String email;

    public Persona(String nome, String email) {
        this.nome = nome;
        this.email = email;
    }
}
```

SUPER()

`super()` invoca il costruttore della superclasse: si chiama una sola volta, prima di usare l'oggetto

- `super(nome, email)` *chiama* `Persona(String, String)`
- *Poi si inizializzano i campi propri della sottoclasse*
- *Fino a Java 21 deve essere la prima istruzione; da Java 25 può esserci codice prima*

```
class Studente extends Persona {
    private String matricola;

    public Studente(String nome, String email,
                    String matricola) {
        super(nome, email);     // inizializza la parte Persona
        this.matricola = matricola;
    }
}
```

SUPER(): JAVA 21 E JAVA 25

Da Java 25 prima di `super()` si può scrivere codice, purché non usi l'oggetto in costruzione

- *Prima di* `super()` *non si leggono i campi e non si chiamano metodi dell'oggetto*
- *Compilato per Java 21, il codice a destra dà errore*

```
// Java 21: super è la prima istruzione
public Studente(String nome, String matricola) {
    super(nome.trim());
    this.matricola = matricola;
}
```

```
// Java 25: si può preparare il valore prima
public Studente(String nome, String matricola) {
    String pulito = nome.trim();
    super(pulito);
    this.matricola = matricola;
}
```

SUPER() IMPLICITO

Se non si scrive `super()`, Java inserisce automaticamente `super()` senza argomenti

- *Funziona solo se la superclasse ha un costruttore senza parametri*
- *Se la superclasse ha solo costruttori con parametri:* ***errore di compilazione***
- *Regola: scrivere sempre* `super()` *in modo esplicito per chiarezza*

```
class Persona {
    // nessun costruttore → Java genera Persona()
}

class Studente extends Persona {
    public Studente() {
        // Java inserisce: super();
    }
}
```

ERRORE COMUNE: SUPER MANCANTE

Se la superclasse non ha un costruttore senza parametri e la sottoclasse non chiama `super(...)`, il compilatore segnala un errore

```
class Persona {
    public Persona(String nome, String email) { ... }
    // nessun Persona() senza parametri!
}

class Studente extends Persona {
    public Studente(String matricola) {
        // ERRORE: il compilatore prova super()
        // ma Persona() non esiste
        this.matricola = matricola;
    }
}
```

Terminale

```
error: constructor Persona
  in class Persona cannot be
  applied to given types;
  required: String,String
  found:    no arguments
```

CATENA DI COSTRUTTORI

Ogni costruttore chiama quello della sua superclasse: i corpi terminano dal più generale al più specifico

- *La chiamata parte da `Studente` e risale fino a `Object`*
- *Ogni livello inizializza i propri campi*
- *Quando il costruttore termina, l'oggetto è completo*

```
// Ordine in cui terminano i costruttori:
// 1. Object()              ← radice
// 2. Persona(nome, email)  ← superclasse
// 3. Studente(nome, email, matricola)

Studente s = new Studente("Anna", "a@b.it", "M001");
```

SUPER.METODO()

`super` non serve solo per i costruttori: si può usare per invocare un **metodo della superclasse**

- `super.toString()` *chiama il* `toString()` *di Persona*
- *Utile per* ***estendere*** *il comportamento senza riscriverlo*
- *Senza* `super.`*, il metodo chiamerebbe se stesso (ricorsione infinita)*

```
class Studente extends Persona {
    private String matricola;

    @Override
    public String toString() {
        return super.toString()
             + " - Matricola: " + matricola;
    }
}
```

IL MODIFICATORE PROTECTED

`protected` rende un membro visibile nella sottoclasse e nel package, ma non all'esterno

- *In pratica: usare* `private` *+ getter è la scelta più comune*
- `protected` *si usa quando la sottoclasse deve accedere direttamente*

```
class Persona {
    protected String nome;  // visibile nelle sottoclassi
    private String email;    // solo dentro Persona
}

class Studente extends Persona {
    public void stampa() {
        System.out.println(nome);  // OK: protected
        System.out.println(email); // ERRORE: private
    }
}
```

I MODIFICATORI DI ACCESSO

Riepilogo completo della visibilità in Java

- *La visibilità cresce dall'alto verso il basso*
- `protected` *aggiunge solo l'accesso dalle sottoclassi rispetto al default*

Modificatore	Stessa classe	Stesso package	Sottoclasse	Ovunque
`private`	Sì	No	No	No
(nessuno)	Sì	Sì	No	No
`protected`	Sì	Sì	Sì	No
`public`	Sì	Sì	Sì	Sì

ESEMPIO COMPLETO

Una gerarchia Persona-Studente-Docente con costruttori e super

```
class Persona {
    private String nome;
    private String email;
    public Persona(String nome, String email) {
        this.nome = nome;
        this.email = email;
    }
    public String getNome() { return nome; }
    public String getEmail() { return email; }
    @Override
    public String toString() {
        return nome + " (" + email + ")";
    }
}
```

```
class Studente extends Persona {
    private String matricola;
    public Studente(String nome, String email, String matricola) {
        super(nome, email);
        this.matricola = matricola;
    }
    public String getMatricola() { return matricola; }
}
```

QUIZ

Il codice compila? Se no, perché?

```
class Veicolo {
    private String targa;
    public Veicolo(String targa) {
        this.targa = targa;
    }
}

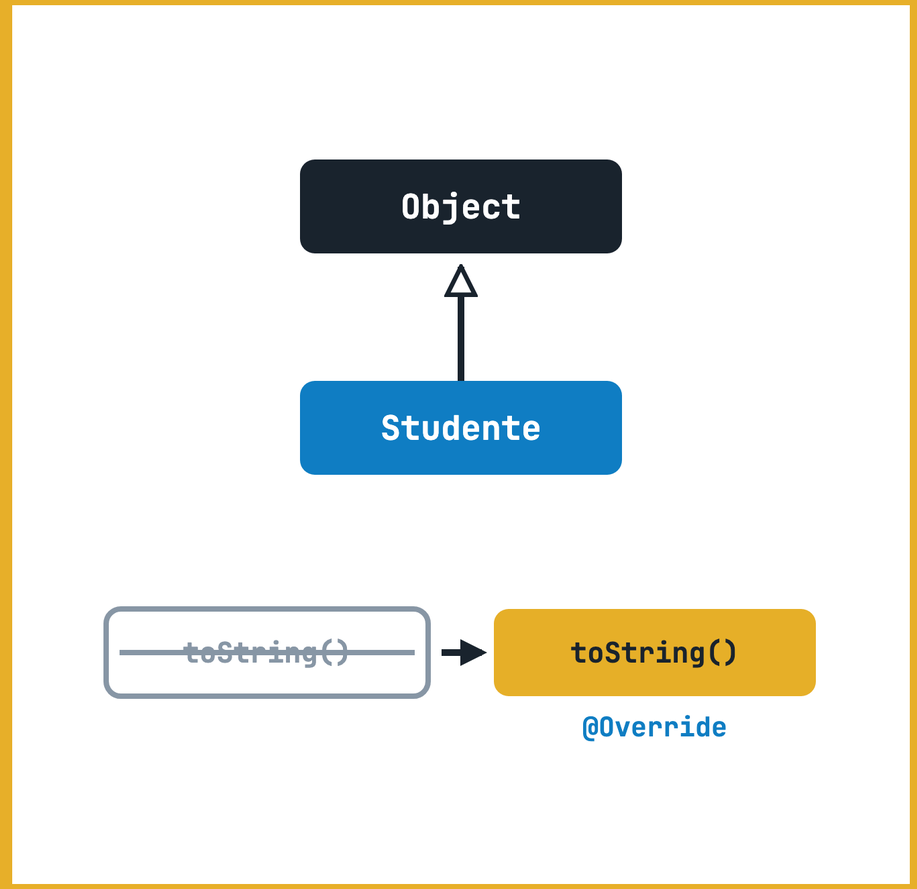
class Auto extends Veicolo {
    private int porte;
    public Auto(String targa, int porte) {
        this.porte = porte;
    }
}
```

RISPOSTE

- ***No**, il codice non compila*
- *Il costruttore di `Auto` non chiama `super(targa)`*
- *Java inserisce `super()` senza argomenti, ma `Veicolo()` non esiste*
- *Il compilatore segnala: constructor Veicolo cannot be applied to given types*

```
// Correzione:
public Auto(String targa, int porte) {
    super(targa);  // chiama Veicolo(targa)
    this.porte = porte;
}
```

03
OVERRIDE E OBJECT



COS'È L'OVERRIDE

Una sottoclasse può **ridefinire** un metodo ereditato per cambiarne il comportamento

- *Il metodo nella sottoclasse **sostituisce** quello della superclasse*
- *Si dice che la sottoclasse fa l'**override** (sovrascrittura) del metodo*

```
class Persona {
    @Override
    public String toString() {
        return nome + " (" + email + ")";
    }
}

class Studente extends Persona {
    @Override
    public String toString() {
        return super.toString()
             + " - Matricola: " + matricola;
    }
}
```

L'ANNOTAZIONE @OVERRIDE

`@Override` dice al compilatore: "sto ridefinendo un metodo della superclasse, verifica che esista"

- *Se il metodo non esiste nella superclasse, il compilatore segnala l'errore*
- *Protegge da errori di battitura nel nome del metodo*
- *Buona pratica: usare **sempre** `@Override`*

```
// senza @Override: compila ma non fa override
public String tostring() {   // "s" minuscola: bug!
    return nome;
}

// con @Override: il compilatore segnala l'errore
@Override
public String tostring() {   // ERRORE: non esiste
    return nome;              // nella superclasse
}
```

LE REGOLE DELL'OVERRIDE

Per fare correttamente l'override di un metodo, occorre rispettare queste regole

- *Se si cambiano i parametri, non è override ma **overloading***
- *Un metodo* `public` *non può diventare* `private`

Requisito	Dettaglio
Stesso nome	Il metodo deve avere lo stesso nome
Stessi parametri	Numero, tipo e ordine dei parametri identici
Tipo di ritorno	Stesso tipo o un suo sottotipo (covariante)
Visibilità	Uguale o più ampia (mai più restrittiva)

OVERRIDE IN AZIONE

Java sceglie il metodo in base al **tipo reale** dell'oggetto, non al tipo della variabile

- `p2` *è dichiarato come* `Persona` *ma è uno* `Studente`
- *Java invoca il* `toString()` *di Studente (tipo reale)*
- *Questo meccanismo si chiama* ***polimorfismo*** *(prossime lezioni)*

```
Persona p1 = new Persona("Anna", "a@b.it");
Persona p2 = new Studente("Marco", "m@b.it", "M01");

System.out.println(p1.toString());
// → Anna (a@b.it)

System.out.println(p2.toString());
// → Marco (m@b.it) - Matricola: M01
```

LA CLASSE OBJECT

`java.lang.Object` è la superclasse di tutte le classi Java. Ogni oggetto eredita i suoi metodi

- *L'implementazione di default è spesso inutile: va ridefinita*
- `hashCode()` *si approfondisce con le collezioni*

Metodo	Descrizione
`toString()`	Restituisce una rappresentazione testuale dell'oggetto
`equals(Object)`	Confronta due oggetti per uguaglianza logica
`hashCode()`	Restituisce un codice numerico (legato a equals)
`getClass()`	Restituisce la classe dell'oggetto a runtime

TOSTRING() DI DEFAULT

Il `toString()` ereditato da Object restituisce il nome della classe e un codice hash: poco utile

- `println(obj)` *chiama automaticamente* `obj.toString()`
- *Ridefinire* `toString()` *per avere un output leggibile*

```
class Persona {
    private String nome;
    public Persona(String nome) { this.nome = nome; }
    // nessun toString() ridefinito
}

Persona p = new Persona("Anna");
System.out.println(p);
```

Terminale

```
Persona@1b6d3586
```

TOSTRING() RIDEFINITO

Un buon `toString()` restituisce i dati significativi dell'oggetto

```
class Persona {
    private String nome;
    private String email;
    public Persona(String nome, String email) {
        this.nome = nome;
        this.email = email;
    }

    @Override
    public String toString() {
        return nome + " (" + email + ")";
    }
}

Persona p = new Persona("Anna", "a@b.it");
System.out.println(p);
```

Terminale

```
Anna (a@b.it)
```

OVERRIDE VS OVERLOADING

Due concetti diversi che spesso si confondono

	Override	**Overloading**
Dove	Nella sottoclasse	Nella stessa classe
Nome	Stesso	Stesso
Parametri	Identici	Diversi
Tipo di ritorno	Stesso o covariante	Qualsiasi
Annotazione	`@Override`	Nessuna
Scopo	Cambiare il comportamento	Offrire varianti
Decisione	A runtime (tipo reale)	A compilazione (tipo parametri)

`EQUALS()`: CENNO

L'`equals()` di Object confronta i riferimenti (come `==`). Per confrontare il contenuto, va ridefinito

- `equals()` *si ridefinisce per il confronto logico, come nell'esercizio della Lampadina*
- *Con le collezioni (*`HashMap`, `HashSet`*) si vedrà perché serve* `hashCode()`
- *Se si ridefinisce* `equals()`*, bisogna ridefinire anche* `hashCode()`

```
Persona p1 = new Persona("Anna", "a@b.it");
Persona p2 = new Persona("Anna", "a@b.it");

System.out.println(p1 == p2);      // false (riferimenti)
System.out.println(p1.equals(p2)); // false (default Object)
```

FINAL: BLOCCARE L'EREDITARIETÀ

`final` impedisce l'override di un metodo o l'estensione di una classe

- `String` *è una classe* `final`*: non la si può estendere*

```
// final su un METODO: non si può ridefinire
class Persona {
    public final String getNome() { return nome; }
}

class Studente extends Persona {
    @Override
    public String getNome() { ... } // ERRORE!
}

// final su una CLASSE: non si può estendere
final class Configurazione { ... }

class MiaConfig extends Configurazione { } // ERRORE!
```

ESEMPIO: GERARCHIA ANIMALE

Una seconda gerarchia per esercitarsi: Animale, Cane, Gatto

```
class Animale {
    private String nome;
    public Animale(String nome) { this.nome = nome; }
    public String getNome() { return nome; }
    public String verso() { return "..."; }
    @Override
    public String toString() { return nome; }
}
```

```
class Cane extends Animale {
    private String razza;
    public Cane(String nome, String razza) {
        super(nome);
        this.razza = razza;
    }
    @Override
    public String verso() { return "Bau!"; }
    @Override
    public String toString() {
        return super.toString() + " (" + razza + ")";
    }
}
```

ESEMPIO: GATTO E MAIN

- *Ogni sottoclasse ridefinisce* `verso()` *con il proprio comportamento*
- *Java invoca il metodo giusto in base al tipo reale dell'oggetto*

```
class Gatto extends Animale {
    private boolean indoor;
    public Gatto(String nome, boolean indoor) {
        super(nome);
        this.indoor = indoor;
    }
    @Override
    public String verso() { return "Miao!"; }
}

// nel main
Animale a1 = new Cane("Rex", "Pastore Tedesco");
Animale a2 = new Gatto("Micio", true);

System.out.println(a1 + " fa " + a1.verso());
// Rex (Pastore Tedesco) fa Bau!
System.out.println(a2 + " fa " + a2.verso());
// Micio fa Miao!
```

QUIZ

Cosa stampa il seguente codice?

- *A)* `Ciao da A`
- *B)* `Ciao da B`
- *C) Errore di compilazione*

```
class A {
    public String saluta() { return "Ciao da A"; }
}

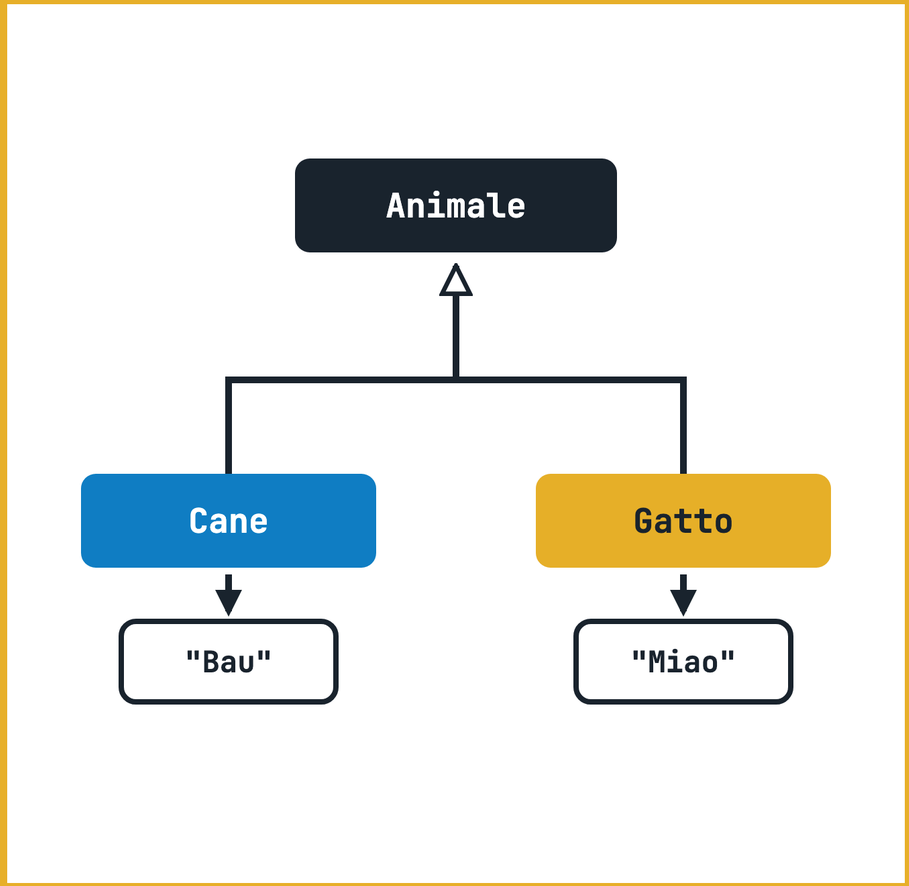
class B extends A {
    @Override
    public String saluta() { return "Ciao da B"; }
}

A obj = new B();
System.out.println(obj.saluta());
```

RISPOSTE

- *Risposta: **B) Ciao da B***
- *La variabile `obj` è dichiarata come `A`*
- *Ma l'oggetto reale è di tipo `B` (`new B()`)*
- *Java invoca il metodo del **tipo reale**, non del tipo dichiarato*
- *Questo è il principio del **polimorfismo***

04
EREDITARIETÀ IN PRATICA



QUANDO USARE L'EREDITARIETÀ

Il test fondamentale: la sottoclasse **"è un"** tipo della superclasse?

- *Se il test fallisce, probabilmente serve la **composizione***

Relazione	Test "è un"	Ereditarietà
Studente - Persona	Uno studente è una persona	Sì
Cane - Animale	Un cane è un animale	Sì
Auto - Veicolo	Un'auto è un veicolo	Sì
Motore - Auto	Un motore è un'auto?	No
Aula - Scuola	Un'aula è una scuola?	No

COMPOSIZIONE: "HA UN"

Se la relazione è **"ha un"**, si usa la composizione: un oggetto contiene un altro come campo

- *La composizione è più flessibile: si può cambiare il motore a runtime*
- *L'ereditarietà crea un legame fisso e permanente*

```
// SBAGLIATO: un'auto non "è un" motore
class Auto extends Motore { }

// CORRETTO: un'auto "ha un" motore
class Auto {
    private Motore motore;

    public Auto(Motore motore) {
        this.motore = motore;
    }
}
```

EREDITARIETÀ VS COMPOSIZIONE

Due strumenti diversi per problemi diversi

- *Regola pratica: "preferire la composizione all'ereditarietà"*
- *Usare l'ereditarietà solo quando la relazione "è un" è genuina*

	Ereditarietà	**Composizione**
Relazione	"è un" (is-a)	"ha un" (has-a)
Legame	Fisso a compilazione	Flessibile a runtime
Riuso	Si ereditano tutti i metodi	Si delega caso per caso
Accoppiamento	Forte	Debole
Esempio	Studente extends Persona	Auto ha un Motore

CONFRONTO CON PYTHON

L'ereditarietà esiste anche in Python con differenze importanti

- *Il concetto è lo stesso; cambia la sintassi e i vincoli*

	Java	Python
Sintassi	`class B extends A`	`class B(A):`
Ereditarietà multipla	No (solo interfacce)	Sì
Superclasse	`super.metodo()`	`super().metodo()`
Costruttore	`super(args)`	`super().__init__(args)`
Override	`@Override`	Implicito (nessuna annotazione)
Visibilità	`private`, `protected`, `public`	Convenzione `_` e `__`

ERRORI COMUNI

Gli errori più frequenti quando si usa l'ereditarietà

Errore	Causa	Correzione
Oggetto usato prima di `super()`	`this` o un metodo prima di `super()`	Spostare l'uso dopo `super()`
Costruttore padre non trovato	Manca super(args) esplicito	Aggiungere super(args) corretto
Override non funziona	Parametri diversi (overloading)	Usare @Override per controllo
Campo private non accessibile	Accesso diretto a campo padre	Usare getter o protected
Ereditarietà al posto di composizione	Relazione "ha un" non "è un"	Usare un campo invece di extends

RIEPILOGO

- ***Ereditarietà*** — *una sottoclasse estende una superclasse con* `extends`
- ***Cosa si eredita*** — *campi e metodi non* `private`*; i costruttori no*
- `super()` — *invoca il costruttore della superclasse (in Java 21 è la prima istruzione)*
- ***super.metodo()*** — *invoca un metodo della superclasse*
- ***Override*** — *ridefinire un metodo ereditato; usare sempre* `@Override`
- ***Object*** — *superclasse di tutto; ridefinire almeno* `toString()`
- ***final*** — *blocca l'override (metodo) o l'estensione (classe)*
- ***"è un" vs "ha un"*** — *ereditarietà vs composizione*

QUIZ

Quale relazione va modellata con l'ereditarietà e quale con la composizione?

- *A)* `Cerchio` *e* `Forma`
- *B)* `Biblioteca` *e* `Libro`
- *C)* `Professore` *e* `Persona`
- *D)* `Computer` *e* `Processore`
- *E)* `Triangolo` *e* `Forma`

RISPOSTE

Relazione	Tipo	Motivo
Cerchio - Forma	Ereditarietà	Un cerchio **è** una forma
Biblioteca - Libro	Composizione	Una biblioteca **ha** dei libri
Professore - Persona	Ereditarietà	Un professore **è** una persona
Computer - Processore	Composizione	Un computer **ha** un processore
Triangolo - Forma	Ereditarietà	Un triangolo **è** una forma

DOMANDE?



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.1